

QuickNote

Dokumentacja techniczna aplikacji mobilnej

Aplikacja Android do zarządzania notatkami z kategoriami, priorytetami i przypomnieniami

Wersja dokumentu: 1.0

Wersja aplikacji: 1.0 (versionCode 1)

Spis treści

Spis treści.....	2
1. Wprowadzenie	3
1.1 Główne funkcjonalności	3
1.2 Platforma i wymagania	3
2. Architektura aplikacji	3
2.1 Warstwy architektury.....	3
2.2 Przepływ danych.....	3
2.3 Kluczowe biblioteki i komponenty	4
3. Model danych	4
3.1 Encja Note (tabela „notes”)	4
3.2 Encja Category (tabela „categories”).....	4
3.3 NoteDao — operacje na notatkach	4
3.4 CategoryDao — operacje na kategoriach	4
4. Warstwa logiki — NoteViewModel	4
5. Interfejs użytkownika — ekrany.....	5
5.1 Ekran główny — MainActivity	5
5.2 Ekran dodawania/edycji — AddEditNoteActivity	5
5.3 Ekran ustawień — SettingsActivity	5
6. Przypomnienia i powiadomienia	5
6.1 AlarmHelper	5
6.2 ReminderReceiver	6
6.3 BootReceiver	6
7. Motyw aplikacji — ThemeHelper	6
8. Uprawnienia aplikacji.....	6
9. Struktura projektu	6
10. Kompilacja i uruchomienie.....	6
11. Możliwe kierunki rozwoju.....	7

1. Wprowadzenie

QuickNote to natywna aplikacja mobilna na platformę Android, umożliwiająca szybkie tworzenie, edycję i organizację notatek. Aplikacja pozwala na przypisywanie notatek do własnych, kolorowych kategorii, oznaczanie ich priorytetem oraz ustawianie przypomnień wyzwalanych o zaplanowanej porze, nawet po ponownym uruchomieniu urządzenia.

Dokument ma na celu opisanie architektury, struktury danych, funkcjonalności oraz technicznych aspektów implementacji aplikacji na podstawie kodu źródłowego projektu.

1.1 Główne funkcjonalności

- Tworzenie, edycja i usuwanie notatek (tytuł, treść, kategoria, priorytet)
- Organizacja notatek w kategoriach z własną nazwą i kolorem
- Oznaczanie notatek jako „ważne” (priorytet)
- Wyszukiwanie notatek w czasie rzeczywistym po tytule i treści
- Filtrowanie listy notatek po kategorii (chipy)
- Sortowanie listy: według daty, priorytetu lub tytułu (A-Z)
- Przypomnienia o notatkach z wyborem daty i godziny, realizowane jako powiadomienia systemowe
- Przywracanie zaplanowanych przypomnień po restarcie urządzenia
- Usuwanie notatki gestem przesunięcia (swipe) z możliwością cofnięcia akcji (Snackbar)
- Udostępnianie treści notatki do innych aplikacji
- Zmiana motywu aplikacji: jasny / ciemny / systemowy
- Usuwanie wszystkich notatek jednym przyciskiem w ustawieniach

1.2 Platforma i wymagania

Parametr	Wartość
System operacyjny	Android
Minimalna wersja SDK (minSdk)	33 (Android 13)
Docelowa wersja SDK (targetSdk)	36
Kompilacja SDK (compileSdk)	36
Język programowania	Kotlin
Wersja Kotlin	2.2.10
Android Gradle Plugin	9.2.1
Zgodność źródeł Java	11
Identyfikator aplikacji	com.example.quicknote

2. Architektura aplikacji

Aplikacja zbudowana jest w oparciu o wzorzec architektoniczny MVVM (Model-View-ViewModel), zalecany przez Google dla aplikacji Android. Warstwy aplikacji są wyraźnie rozdzielone, co ułatwia utrzymanie i testowanie kodu.

2.1 Warstwy architektury

Warstwa	Odpowiedzialność	Kluczowe klasy
UI (View)	Wyświetlanie danych, obsługa interakcji użytkownika	MainActivity, AddEditNoteActivity, SettingsActivity, NoteAdapter, CategoryAdapter
ViewModel	Logika prezentacji, filtrowanie, sortowanie, komunikacja z repozytorium	NoteViewModel, NoteViewModelFactory
Repository	Pośrednik między ViewModel a źródłem danych	NoteRepository
Data (Model)	Dostęp do bazy danych, definicje encji	AppDatabase, NoteDao, CategoryDao, Note, Category

2.2 Przepływ danych

Dane przepływają jednokierunkowo: warstwa UI obserwuje obiekty LiveData eksponowane przez NoteViewModel. Zmiany filtrów (wyszukiwanie, kategoria, sortowanie) są łączone za pomocą MediatorLiveData i funkcji switchMap, co automatycznie odświeża listę notatek przy każdej zmianie dowolnego z tych parametrów. Operacje zapisu (insert/update/delete) są uruchamiane w wątku IO za pośrednictwem NoteRepository, aby nie blokować wątku głównego.

2.3 Kluczowe biblioteki i komponenty

Biblioteka	Zastosowanie
Room (2.8.4)	Lokalna baza danych SQLite (ORM), DAO, LiveData
Lifecycle / ViewModel / LiveData (2.10.0)	Zarządzanie stanem UI odpornym na zmiany konfiguracji
Kotlin Coroutines	Operacje asynchroniczne na bazie danych (Dispatchers.IO)
Material Components (1.14.0)	Komponenty UI: Chip, Snackbar, TextInputEditText, MaterialCardView
RecyclerView (1.4.0) + ListAdapter/DiffUtil	Wydajne listy notatek i kategorii
AlarmManager (Android SDK)	Planowanie precyzyjnych przypomnień (setAlarmClock)
NotificationCompat	Budowanie i wyświetlanie powiadomień systemowych
KSP	Przetwarzanie adnotacji Room w czasie kompilacji

3. Model danych

Aplikacja korzysta z lokalnej bazy danych Room o nazwie „note_database” (wersja schematu: 2), zawierającej dwie tabele: notes oraz categories. W przypadku zmiany schematu bazy stosowana jest strategia fallbackToDestructiveMigration (dane są usuwane i tworzone od nowa).

3.1 Encja Note (tabela „notes”)

Pole	Typ	Opis
id	Long	Klucz główny, generowany automatycznie
title	String	Tytuł notatki
content	String	Treść notatki
categoryId	Long	Odwołanie do Category.id (0 = brak kategorii)
priority	Int	0 = zwykła, 1 = ważna
createdAt	Long	Znacznik czasu utworzenia (ms), domyślnie bieżący czas
reminderTime	Long	Zaplanowany czas przypomnienia (ms); 0 = brak przypomnienia

3.2 Encja Category (tabela „categories”)

Pole	Typ	Opis
id	Long	Klucz główny, generowany automatycznie
name	String	Nazwa kategorii
colorHex	String	Kolor kategorii w formacie szesnastkowym, np. #EF4444

Obie encje implementują interfejs Serializable, co pozwala przekazywać je bezpośrednio w Intent (np. przy otwieraniu ekranu edycji lub w treści powiadomienia).

3.3 NoteDao — operacje na notatkach

Metoda	Opis
getAllNotes()	Zwraca LiveData<List<Note>> posortowaną malejąco po createdAt
insertNote(note)	Wstawia notatkę (REPLACE przy konflikcie ID), zwraca nowe ID
updateNote(note)	Aktualizuje istniejącą notatkę
deleteNote(note)	Usuwa notatkę
searchNotes(query, categoryId)	Wyszukuje po tytule/treści z filtrem kategorii; obsługuje specjalny przypadek kategorii „Inne” (categoryId ≤ 0)
deleteAllNotes()	Usuwa wszystkie notatki
getFutureReminders(currentTime)	Zwraca notatki z reminderTime w przyszłości — używane po restarcie systemu

3.4 CategoryDao — operacje na kategoriach

Metoda	Opis
getAllCategories()	Zwraca LiveData<List<Category>> posortowaną alfabetycznie
insertCategory(category)	Dodaje nową kategorię, zwraca ID
deleteCategoryAndClearNotes(category)	Operacja transakcyjna: najpierw ustawia categoryId=-1 we wszystkich powiązanych notatkach, następnie usuwa kategorię

4. Warstwa logiki — NoteViewModel

Klasa NoteViewModel (dziedzicząca po AndroidViewModel) jest centralnym punktem logiki biznesowej UI. Udostępnia trzy niezależne, obserwowalne parametry:

- `_searchQuery` — aktualna fraza wyszukiwania
- `_filterCategory` — ID wybranej kategorii filtra (-1 = wszystkie)
- `_sortOption` — wybrany tryb sortowania (BY_DATE, BY_PRIORITY, BY_TITLE)

Zmiany dowolnego z powyższych parametrów są łączone przez MediatorLiveData (combinedFilter) w krotkę Triple, a następnie przekształcane operacją switchMap w zapytanie do repozytorium. Wynik zapytania jest dodatkowo sortowany w pamięci zgodnie z wybraną opcją sortowania.

ViewModel udostępnia także metody insert, update, delete, deleteAll, addCategory oraz deleteCategory, uruchamiane w viewModelScope (korutyny). Operacja delete dodatkowo anuluje ewentualny zaplanowany alarm powiązany z notatką.

5. Interfejs użytkownika — ekrany

5.1 Ekran główny — MainActivity

Wyświetla listę notatek w komponencie RecyclerView z użyciem NoteAdapter (ListAdapter + DiffUtil).

Główne elementy ekranu:

- Pasek chipów kategorii (ChipGroup) do filtrowania listy, generowany dynamicznie na podstawie danych z bazy, z dodatkowym stałym chipem „Wszystkie”
- Pole wyszukiwania (SearchView) w pasku menu, filtrujące listę w czasie rzeczywistym
- Menu sortowania z zaznaczeniem aktualnie wybranej opcji
- Przycisk FAB (pływający przycisk akcji) otwierający ekran dodawania nowej notatki
- Obsługa gestu przesunięcia w lewo (ItemTouchHelper) do usuwania notatki, z możliwością cofnięcia akcji przez Snackbar
- Długie kliknięcie na notatce otwiera dialog z opcjami: Edytuj / Udostępnij / Usuń
- Długie kliknięcie na chipie kategorii pozwala ją usunąć (z potwierdzeniem)
- Widok pustej listy (textViewEmpty) wyświetlany, gdy brak notatek
- Obsługa wcięć systemowych (edge-to-edge) dla paska narzędzi i przycisku FAB
- Prośba o uprawnienie POST_NOTIFICATIONS na urządzeniach z Androidem 13+

5.2 Ekran dodawania/edycji — AddEditNoteActivity

Formularz umożliwiający utworzenie nowej notatki lub edycję istniejącej (przekazywanej przez Intent jako obiekt Serializable pod kluczem EXTRA_NOTE). Zawiera:

- Pola tekstowe: tytuł i treść notatki
- Poziomą listę kategorii (CategoryAdapter) z możliwością zaznaczenia jednej kategorii oraz dodania nowej poprzez dedykowany element „+” na końcu listy
- Dialog dodawania kategorii z wyborem nazwy i jednego z sześciu predefiniowanych kolorów
- Przełącznik priorytetu (SwitchMaterial) — oznaczenie notatki jako ważnej
- Przycisk ustawienia przypomnienia, otwierający kaskadowo DatePickerDialog, a następnie TimePickerDialog
- Zapis notatki (walidacja: tytuł nie może być pusty) oraz automatyczne zaplanowanie lub anulowanie alarmu w zależności od ustawionego czasu przypomnienia
- Funkcję udostępniania treści notatki (widoczna w menu tylko w trybie edycji)

5.3 Ekran ustawień — SettingsActivity

- Wybór motywu aplikacji: Jasny / Ciemny / Systemowy (MaterialAutoCompleteTextView)
- Przycisk usuwający wszystkie notatki (z oknem potwierdzenia)
- Wyświetlanie numeru wersji aplikacji, odczytanego z PackageManager

6. Przypomnienia i powiadomienia

Mechanizm przypomnień oparty jest na systemowym AlarmManager oraz mechanizmie powiadomień Android.

6.1 AlarmHelper

- scheduleAlarm(context, note) — planuje alarm typu AlarmClock (najwyższy priorytet budzenia systemu) tylko dla przyszłych dat; na Androidzie 12+ sprawdza uprawnienie canScheduleExactAlarms()
- cancelAlarm(context, noteId) — anuluje istniejący alarm dla danej notatki (jeśli istnieje)

Alarm identyfikowany jest unikalnym PendingIntent na podstawie ID notatki, a obiekt notatki przekazywany jest w całości jako dodatek Intentu (Serializable).

6.2 ReminderReceiver

Odbiornik wywoływany w zaplanowanym momencie przez AlarmManager. Tworzy kanał powiadomień „reminders” (wymagany od Androida 8.0) i wyświetla powiadomienie o wysokim priorytecie z wibracją. Kliknięcie powiadomienia otwiera ekran edycji notatki (AddEditNoteActivity) z przekazanym obiektem notatki.

6.3 BootReceiver

Nasłuchuje zdarzenia systemowego BOOT_COMPLETED. Po ponownym uruchomieniu urządzenia pobiera z bazy danych wszystkie notatki z przyszłym czasem przypomnienia (NoteDao.getFutureReminders) i planuje dla nich alarmy ponownie, ponieważ system Android usuwa wszystkie zaplanowane alarmy po restarcie.

7. Motyw aplikacji — ThemeHelper

Obiekt singleton zarządzający trybem wyświetlania (jasny/ciemny/systemowy) z wykorzystaniem AppCompatActivity.setDefaultNightMode. Preferencja użytkownika zapisywana jest w SharedPreferences (plik „quicknote_prefs”, klucz „theme_mode”) i odczytywana przy starcie aplikacji w klasie QuickNoteApplication, dzięki czemu wybrany motyw jest stosowany natychmiast po uruchomieniu, przed wyświetleniem jakiegokolwiek ekranu.

8. Uprawnienia aplikacji

Uprawnienie	Cel
POST_NOTIFICATIONS	Wyświetlanie powiadomień o przypomnieniach (wymagane od Androida 13)
SCHEDULE_EXACT_ALARM	Planowanie precyzyjnych alarmów przypomnień
RECEIVE_BOOT_COMPLETED	Przywracanie zaplanowanych przypomnień po restarcie urządzenia

9. Struktura projektu

app/src/main/java/com/example/quicknote/

```
├─ MainActivity.kt          – ekran główny, lista notatek
├─ AddEditNoteActivity.kt   – dodawanie/edycja notatki
├─ SettingsActivity.kt     – ustawienia aplikacji
├─ AlarmHelper.kt          – planowanie/anulowanie alarmów
├─ ReminderReceiver.kt     – obsługa wyzwolonego przypomnienia
├─ BootReceiver.kt         – przywracanie alarmów po restarcie
├─ QuickNoteApplication.kt – inicjalizacja motywu przy starcie
├─ data/
│   └─ Note.kt             – encja notatki
│   └─ Category.kt         – encja kategorii
│   └─ NoteDao.kt / CategoryDao.kt
│   └─ AppDatabase.kt      – konfiguracja bazy Room
│   └─ NoteRepository.kt   – warstwa pośrednia dostępu do danych
├─ ui/
│   └─ NoteViewModel.kt / NoteViewModelFactory.kt
│   └─ NoteAdapter.kt / CategoryAdapter.kt
└─ ThemeHelper.kt
```

10. Kompilacja i uruchomienie

Projekt jest standardowym modułem Gradle dla systemu Android, zarządzanym przez Android Studio.

- Zbuduj projekt: ./gradlew assembleDebug
- Zainstaluj na podłączonym urządzeniu/emulatorze: ./gradlew installDebug
- Minimalna wersja Androida na urządzeniu docelowym: Android 13 (API 33)
- System zarządzania zależnościami: katalog wersji Gradle (gradle/libs.versions.toml)

11. Możliwe kierunki rozwoju

- Migracja do Room z zdefiniowaną ścieżką migracji zamiast fallbackToDestructiveMigration, aby nie tracić danych przy aktualizacjach schematu
- Wsparcie dla wielu przypomnień na jedną notatkę
- Synchronizacja notatek w chmurze / kopia zapasowa poza urządzeniem
- Testy jednostkowe i instrumentalne (obecnie tylko szablony ExampleUnitTest / ExampleInstrumentedTest)
- Migracja UI do Jetpack Compose (biblioteki Compose są już dołączone w zależnościach, ale nieużywane w bieżących ekranach)